

Assignment 6: Client-Server Messaging

Teachers: Fatima Zahra Errounda, Stephan Schiffel

Document last updated: October 5, 2026

Task and Logistics

Your task is to implement a simple mail server and associated client application using TCP sockets.

- This is a group assignment. You may work in groups of two or three students.
- The programming language is C or C++.
- Your code will be graded on Linux (a standard distribution, similar to the TSAM server). You may develop on another operating system (e.g. macOS, or Windows with WSL), but you are responsible for testing that your code compiles and runs correctly on Linux before submitting it.
- Your code **must not require third-party libraries for networking or socket handling** — all networking code must use the standard C/C++ socket API directly. You may use additional libraries for other purposes, such as string handling or logging, but any such dependency must be included in your submission. If in doubt, ask!
- You may start from scratch, or use the very simple, sample client-server project provided as a base. Note that the sample project has little to do with the protocol you need to implement here **and contains some errors** (see **Hints** below).

Server Specification

- Your server must listen on one TCP port for clients to connect to it.
- The server must be able to maintain connections to several clients simultaneously and react to commands from all these clients without unreasonable delay.
- The server must be able to handle a sequence of multiple requests per connection.
- The server must operate autonomously! The server must not accept nor wait for any input on the terminal. All communication with the server must happen through TCP sockets.
- You may run the server on the TSAM server or on some other computer. Note that in a later assignment, you must run the server such that other groups can connect to it.
- The executable of your server must be called `tsamserver`. The server port you are listening on must be the first command line argument. For example:

```
./tsamserver 4044
```

Any additional command line arguments you want to define must be optional. Your server must function in a test without additional parameters or configuration.

- The server must keep a timestamped log of all commands sent and received. This log may contain other information about the server state to facilitate debugging.
- Do not hard-code any IP addresses or port numbers. Use additional command line arguments, a config file or commands sent from your client if you need additional input.

Client Specification

- The client must connect to your server on a given IP and TCP port.
- The executable of your client must be called `tsamclient`. It must accept two command line arguments: IP address and port of the server. For example:

```
./tsamclient 127.0.0.1 4044
```

Any additional command line arguments you want to define must be optional. Your client must function in a test without additional parameters or configuration.

- The client must accept user input from stdin and as result send commands to the server (commands that need to be accepted are listed below).
- The client must receive responses send by the server and display them for the user.
- The client must print out all commands sent and responses received with a human-readable timestamp (date and time).

Communication Protocol

All commands sent to the server and responses sent to the client must use the following format: <SOH><length><STX><command><ETX> with

- the ASCII characters 0x01 (<SOH>), 0x02 (<STX>), and 0x03 (<ETX>) denoting the start of the header, start of text, and end of text, respectively, and
- `length` being a 16-bit unsigned integer (in network-byte order) denoting the length of the transmission in bytes (including the five bytes for framing the message: <SOH><length><STX> and <ETX>).

Your server must support commands up to a length of 5000 bytes. It may discard any commands that are longer.

Your server must understand the following commands from your client. You may implement additional commands if you wish.

Command	Expected behaviour
SENDMSG,<from>,<to>,<message>	Store the given message for the recipient <to> from the sender <from>
GETMSG,<username>	Return a single stored message for the user with the given name and delete it afterwards
STATUSREQ	Return information for which users have messages stored for them, and how many

The response format for SENDMSG and GETMSG is up to you to choose, as long as you use it consistently between your own client and your own server.

The response to STATUSREQ, however, is **fixed** (so that this part of your server's behaviour can be tested automatically) and must look exactly like this:

```
<username1>:<count1>,<username2>:<count2>,...
```

listing, for every user who currently has at least one stored message, their username and the number of messages currently stored for them, separated by commas, with no spaces. Users may be listed in any order. If no messages are currently stored for any user, the response must be exactly the text EMPTY.

Hints

- TCP provides a byte stream, not a sequence of messages: a single call to `recv()` may return only part of a command, several commands at once, or a command split across multiple calls. Your server (and client) must buffer incoming bytes and only process a command once a complete frame (`<SOH> . . . <ETX>`) has been received. The sample server code provided does **not** do this correctly.
- Avoid busy-waiting. Your server and client must block (e.g. in `poll()`, `select()`, `epoll_wait()`, or a blocking `recv()` running in its own thread) while waiting for input, rather than repeatedly polling in a loop. CPU usage must be essentially zero while idle. Sleep calls are not a solution.
- Make sure you cover all error cases for all the system calls correctly.
- Test your server with several simultaneous client connections, and with clients repeatedly disconnecting and reconnecting, to make sure that your bookkeeping of open connections and stored messages stays consistent.
- Test both client and server with input that does **not** follow the protocol (e.g., using `ncat`). They should log and/or ignore malformed input, but not crash.

Submission

Submit a single zip file containing all source code for your client and server and the following files named exactly:

```
Makefile
README.md
trace.pcapng
server.log
client-output.txt
```

- Makefile: running `make` in the submission directory, with no arguments and no manual setup, must build both of the following executables:
 - `tsamserver` — your server,
 - `tsamclient` — your client.
- README.md must describe:
 - how to compile and run your code,
 - which operating system you developed and tested on,
 - the commands implemented by your client and server, and
 - the behaviour of your server in response to these commands and to other relevant events (e.g. malformed input, clients disconnecting). You should especially describe any additional commands or behaviour that are not already described in this assignment.

The zip file must **not** contain hg/git/etc. repositories, `__MACOSX` or related files/folders, compiled binaries/object files, or other hidden files.

Grading

1. (a) (60 points) Client and server are implemented as described above and
 - handle malformed commands/messages gracefully (ignore them, but don't crash),
 - handle multiple clients being connected simultaneously and react to each of them sending commands,
 - handle clients disconnecting and connecting again,
 - do not busy-wait.
- (b) (10 points) Demonstrating the correct behaviour of client and server by providing:
 - a wireshark trace in `trace.pcapng`,
 - the server log file `server.log`, and
 - the output of the client as text in `client-output.txt`for one session that shows all the implemented commands and responses.

The files must show your client sending the commands and the server responding appropriately. **The wireshark trace must not contain any packets other than those between your client and server.**
- (c) (10 points) The server produces a readable, timestamped log file showing all received and sent commands and other useful information about the internal state and behaviour of the server.
- (d) (10 points) Your submission follows the format described in the **Submission** section above.
- (e) (10 points) Code is well structured and documented.